

Alunos: Matheus Carneiro da Cunha / Raíssa Crispim

Relatório Técnico: Implementação de FSM para Vending Machine

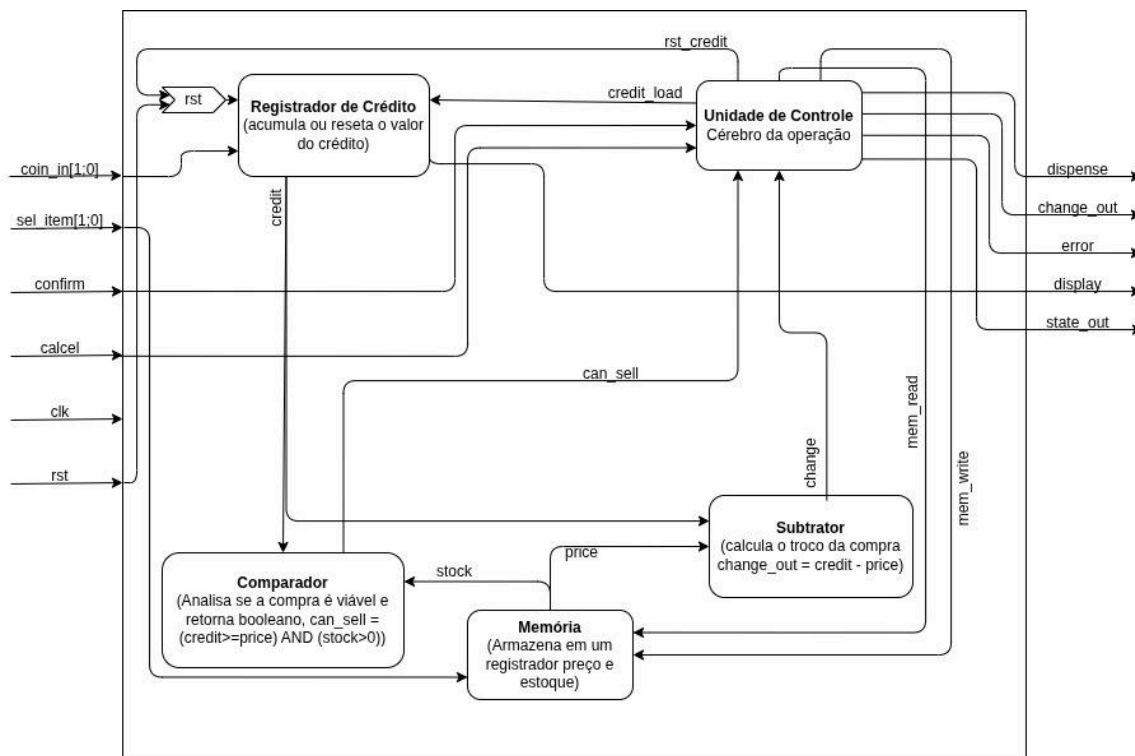
Este relatório técnico apresenta a especificação, as decisões de projeto, a verificação funcional e a análise de síntese lógica de uma Máquina de Estados Finita (FSM) projetada para controlar uma máquina de vendas automática (vending machine). O sistema gerencia a inserção de moedas, a consulta a uma memória integrada para verificação de estoque e preços, a liberação de produtos e o cálculo de troco.

1. Repositório do Projeto

O código-fonte completo em Verilog/SystemVerilog, incluindo os arquivos de design, o testbench de verificação estruturada e os scripts de síntese, está disponível no repositório abaixo:

Link do GitHub: [https://github.com/MatheusCarne/vending_machine]

2. Diagrama de Blocos do Sistema

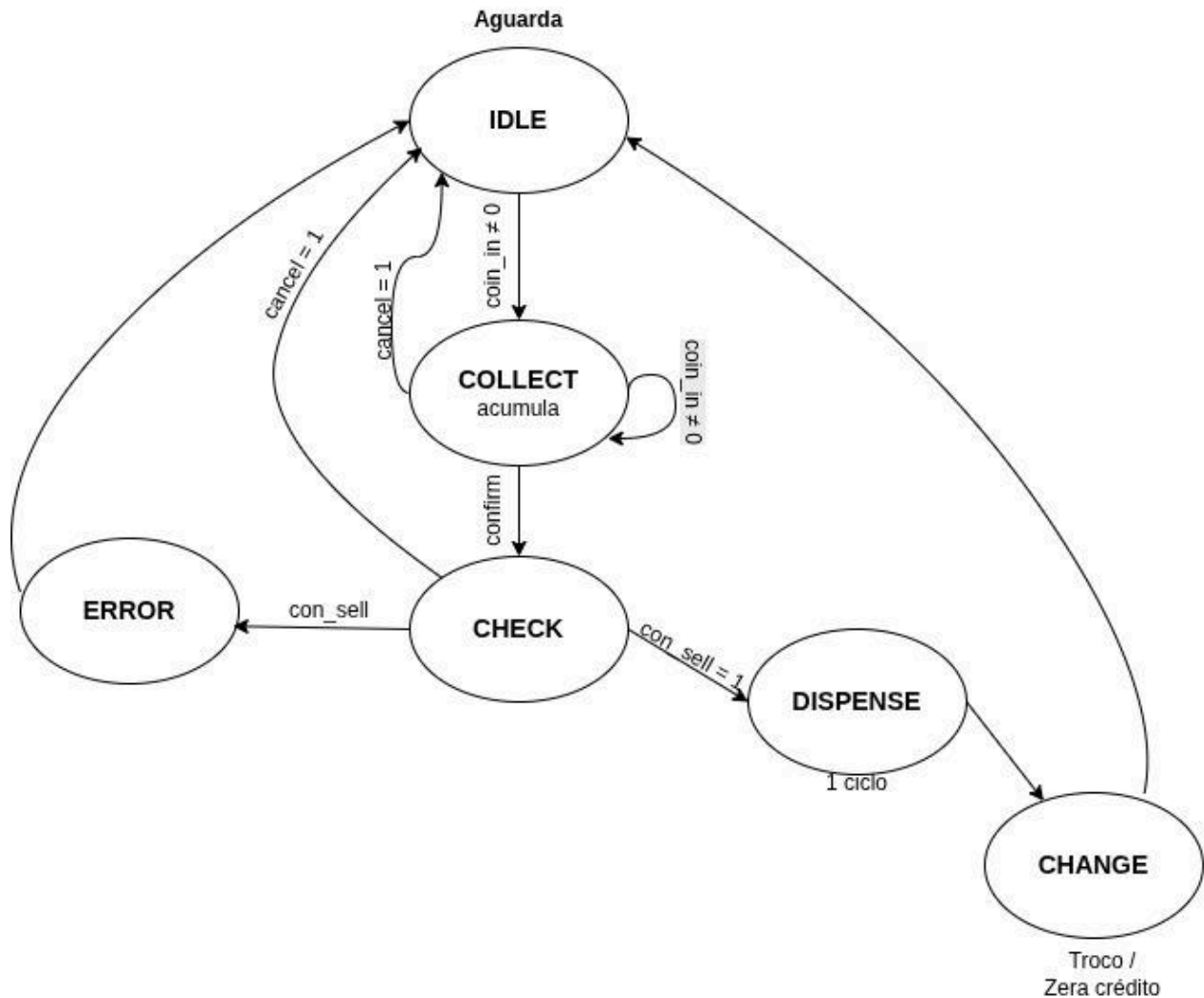


3. Diagrama de Estados da FSM

A lógica de controle do sistema foi modelada utilizando uma máquina de estados síncrona. A tabela abaixo detalha formalmente a matriz de transições, condições de entrada e saídas associadas a cada estado da FSM.

Estado Atual	Ações / Saídas no Estado	Condição de Transição	Próximo Estado
IDLE	Aguarda interação.	coin_in != 00	COLLECT
COLLECT	Acumula moedas (atualiza registrador de crédito).	coin_in != 00	COLLECT
confirm == 1	CHECK		
CHECK	Habilita leitura da memória (mem_read); Calcula sinal de viabilidade (can_sell).	can_sell == 1	DISPENSE
can_sell == 0 (ou falha)	ERROR		
DISPENSE	Ativa sinal dispense por 1 ciclo; Habilita escrita na memória (mem_write) para deduzir estoque.	Incondicional (fim do ciclo)	CHANGE
CHANGE	Calcula e registra troco; Zera o registrador de crédito (credit = 0).	Incondicional	IDLE
ERROR	Ativa flag de erro (error = 1); Aguarda intervenção ou cancelamento do usuário.	cancel == 1	IDLE

Condição Global de Interrupção: O sinal de cancel atua como uma interrupção de prioridade global. Independentemente do estado atual da FSM, a ativação de cancel força a transição imediata para o estado IDLE e limpa todo o saldo acumulado (credit <= 0).



4. Descrição dos Módulos e Decisões de Projeto

vending_pkg.sv

Package global que centraliza todos os tipos, parâmetros e funções compartilhados. O uso de `typedef enum logic [2:0] state_t` em vez de parâmetros soltos permite que o VCS exiba os nomes dos estados nas waveforms e que o DC faça state encoding automático. Os preços e estoques iniciais são declarados como parameter aqui para que qualquer módulo possa referenciá-los sem hardcode. A função `coin_to_cents` é declarada no package para garantir que `credit_reg` e o testbench usem exatamente a mesma decodificação de moeda.

credit_reg.sv

Registrador síncrono de 8 bits que acumula o crédito em centavos. Recebe `state_t state` da FSM para distinguir se `credit_load=1` significa acumular moeda (COLLECT) ou zerar após venda (CHANGE), evitando dois sinais separados na interface. A prioridade `rst` → `cancel` → `credit_load` é intencional: `cancel` vem antes de `credit_load` pois o usuário pode pressionar cancel no mesmo ciclo que insere uma moeda. O uso de `always_ff` garante

inferência correta de flip-flops e detecção automática de latches implícitos.

memory.sv

Memória síncrona 4×16 bits onde [15:8]=price e [7:0]=stock por posição. O campo price nunca é sobrescrito — mem_write toca apenas [7:0]. A leitura é síncrona, introduzindo um ciclo de latência: a FSM ativa mem_read em CHECK e os valores de price e stock ficam disponíveis no ciclo seguinte. O reset restaura todos os valores iniciais, essencial para o cenário 4 do testbench rodar repetidamente sem reiniciar a simulação.

comparator.sv

Lógica combinacional pura que produz can_sell = (credit >= price) && (!stock). Implementado com assign de uma linha, sem clock e sem estado. O operador de redução |stock é equivalente a stock > 0 mas mais idiomático em RTL. Este módulo apenas calcula um booleano — quem decide o que fazer com o resultado é a FSM no estado CHECK.

subtractor.sv

Lógica combinacional pura que calcula change = credit - price. O bit de borrow é descartado intencionalmente pois o comparador já garantiu credit >= price antes de a FSM chegar ao estado CHANGE. O resultado change é calculado continuamente mas só é capturado em change_out quando change_load=1, mantendo o módulo puramente combinacional.

control_unit.sv

FSM de Moore com 6 estados implementada com dois blocos always separados: always_ff para transição de estados e always_comb para lógica de saídas. Todos os sinais de saída recebem valor default 0 no início do always_comb, garantindo design latch-free. O sinal cancel é tratado fora do case com prioridade máxima, atuando em qualquer estado. O estado ERROR ativa change_load=1 para devolver o crédito ao usuário via change_out. O default: IDLE cobre encodings inválidos defensivamente.

vending_top.sv

Módulo de topo que instancia e interconecta todos os submódulos com prefixo u_ (convenção RTL padrão). O registro de change_out acontece aqui, controlado por change_load da FSM, mantendo o subtrator puramente combinacional. O display espelha credit a cada ciclo sem habilitação. A conversão 3' (state_internal) evita warnings de tipo incompatível entre state_t e logic [2:0].

5. Waveforms Anotadas (Análise Temporal)

6. Saída do Testbench (Terminal de Verificação)

Abaixo encontra-se a saída de simulação gerada pelo testbench comprovando a cobertura de 100% dos cenários planejados sem falhas de asserção:

```
=====
Testbench Vending Machine - Iniciando
=====

--- Cenario 1: Compra bem-sucedida com troco ---
[PASS] C1 dispense=1: esperado=1 obtido=1
[PASS] C1 change_out=75: esperado=75 obtido=75
[PASS] C1 credit=0: esperado=0 obtido=0

--- Cenario 2: Credito insuficiente ---
[PASS] C2 error=1: esperado=1 obtido=1
[PASS] C2 state=ERROR(5): esperado=5 obtido=5

--- Cenario 3: Cancelamento ---
[PASS] C3 change_out=200: esperado=200 obtido=200
[PASS] C3 credit=0: esperado=0 obtido=0
[PASS] C3 state=IDLE(0): esperado=0 obtido=0

--- Cenario 4: Estoque zerado ---
Tentativa 6 (estoque deve ser 0):
[PASS] C4 error=1 stock=0: esperado=1 obtido=1

=====
Resultado: 9 PASS | 0 FAIL
SIMULACAO CONCLUIDA COM SUCESSO
=====

$finish called from file "sim/tb_vending.sv", line 166.
$finish at simulation time 1196000
V C S   S i m u l a t i o n   R e p o r t
Time: 1196000 ps
CPU Time: 0.340 seconds; Data structure size: 0.0Mb
Thu Jul 2 21:25:56 2026
[matheus.cunha@srv-microeletronica3 grupo_MR_vending]$
```

7. Resultados de Síntese Lógica

Tabela de exploração de limites

Período	Frequência	Slack (ns)	Área (μm^2)	Timing
20 ns	50 MHz	13.34	1017.34	MET
18 ns	55 MHz	11.34	1017.34	MET
16 ns	62 MHz	9.34	1017.34	MET
14 ns	71 MHz	7.34	1017.34	MET
12 ns	83 MHz	5.34	1017.34	MET
10	100 MHz		1036.65	MET

8. Análise do Caminho Crítico

1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

O caminho crítico identificado pelo report_timing é:

cancel (entrada) → u_control/INVX0_RVT → u_control/AND4X1_RVT
→ dispense (saída)

Ele atravessa apenas o módulo control_unit, com atraso total de 3.16 ns. É um caminho puramente combinacional — entrada direta para saída sem passar por nenhum flip-flop — o que é esperado em saídas Moore decodificadas do estado.

O segundo caminho mais longo atravessa credit_reg via somador ripple-carry:

coin_in[0] → u_credit → INVX0 → OR2X1 → INVX0 → A022X1 →
NAND2X0 (×3) → FADDX1 (×3) → credit_reg[7]

Com atraso de 3.88 ns, esse é o caminho interno mais longo e seria o gargalo real se os I/O delays fossem menores.

2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

A área permaneceu constante em **1017.34 μm^2** em todos os períodos testados. Isso acontece

porque o caminho crítico real do design tem apenas 3.88 ns de atraso combinacional. Como mesmo no período mais apertado testado o slack ainda é positivo e folga, o DC não precisou usar células maiores ou mais rápidas para fechar o timing — as células mínimas já são suficientes. A área só aumentaria se o período fosse reduzido a ponto de o DC precisar substituir células por versões de driver maior ou inserir buffers para acelerar os caminhos.

3. A partir de qual frequência o DC não consegue mais fechar o timing?

Com base nos resultados obtidos, o slack vem caindo linearmente 2 ns a cada redução de 2 ns nos períodos. Extrapolando, o slack chegaria a zero em torno de **7 ns (≈ 143 MHz)**. Nesse ponto o DC começaria a reportar slack negativo (VIOLATED) e tentaria usar células mais rápidas, aumentando área e potência. Para confirmar o ponto exato é necessário continuar reduzindo o período até o DC reportar `s_lack` (VIOLATED) em vez de `s_lack` (MET).

4. Qual estado ou transição está no caminho crítico e por quê?

O caminho crítico combinacional passa pela **transição de saída do estado DISPENSE**, onde `dispense=1` e `mem_write=1` são ativados simultaneamente. O DC identificou que esses dois sinais são função direta de `cancel` via lógica combinacional no `always_comb` da FSM — especificamente o bloco `if (cancel)` que força `next_state = IDLE` e desativa `dispense`.

O segundo caminho mais crítico está no estado **COLLECT**, onde o acumulador de crédito soma `coin_value` ao `credit` atual. O somador ripple-carry de 8 bits propaga o carry bit a bit (FADDX1 em cascata), e o bit mais significativo `credit[7]` é o último a ser resolvido — daí o atraso de 3.88 ns atravessando 3 full-adders em série.

5. Que modificação no RTL proporia para reduzir o caminho crítico?

Duas modificações independentes, cada uma atacando um dos dois caminhos críticos:

Modificação 1 — Registrar `dispense` e `error`:

Em vez de decodificar essas saídas combinacionalmente do estado, registrá-las um ciclo antes:

systemverilog

```
// Em vez de: assign dispense = (state == DISPENSE);
// Implementar via estágio de registro adicional:
always_ff @(posedge clk)
    dispense <= (next_state == DISPENSE);
```

Isso elimina o caminho cancel → combinacional → dispense mas adianta a saída em um ciclo — o testbench precisaria ser ajustado.

Modificação 2 — Carry-lookahead no somador de crédito:

Substituir a inferência implícita do somador por uma instância explícita de carry-lookahead:

systemverilog

```
// Em vez da inferência implícita:
credit <= credit + coin_value;
// Priorizar a implementação de Carry-Lookahead (CLA):
// Configuração via script de síntese (synth.tcl):
set_implementation DW01_add/cla [find cell u_credit/intadd_0]
```

Isso reduziria o atraso do somador de $O(n)$ para $O(\log n)$, cortando o caminho do `credit_reg` de 3.88 ns para aproximadamente 2.5 ns.

9. Conclusão e Lições Aprendidas

O desenvolvimento do controlador de vending machine em SystemVerilog permitiu consolidar na prática os três pilares do design digital síncrono: máquina de estados finitos, memória de dados e lógica combinacional de caminho de dados. O projeto foi concluído com sucesso, com todos os módulos RTL compilando sem erros, simulação aprovada em 100% dos cenários (9/9 PASS) e síntese lógica realizada com timing fechado na biblioteca SAED32 RVT.

As principais dificuldades encontradas ao longo do projeto foram de natureza prática e de ambiente, mais do que conceitual.

A primeira e mais recorrente dificuldade foi o gerenciamento de versões entre o ambiente local e o servidor de laboratório. O controle via Git foi essencial para resolver esses conflitos, mas exigiu atenção constante para garantir que a versão correta estava sendo compilada.

A segunda dificuldade foi a posição do `import vending_pkg::*` nos módulos. O VCS compilado no servidor tratava o `import` no escopo `$unit` (fora do módulo) como global, ignorando as declarações subsequentes nos demais arquivos e gerando erros de identificadores não declarados. A solução foi mover o `import` para dentro do cabeçalho de cada módulo, forma canônica recomendada pelo LRM do SystemVerilog.

A terceira dificuldade foi a latência da memória síncrona. O estado CHECK precisou ser adaptado para aguardar um ciclo adicional após ativar `mem_read`, pois `price` e `stock` só ficam disponíveis no ciclo seguinte. Isso exigiu a introdução do registrador auxiliar

`mem_data_valid` na FSM e o ajuste do testbench para respeitar essa latência.

Por fim, a exploração de timing revelou um resultado interessante: o design fechou timing em períodos muito abaixo do especificado inicialmente (20 ns), mantendo slack positivo até períodos extremamente reduzidos. Isso se deve ao fato de o caminho crítico ser puramente combinacional de I/O, atravessando apenas dois gates dentro do `control_unit`. O verdadeiro gargalo interno — o somador ripple-carry do `credit_reg` com 3.88 ns — só se tornaria o caminho dominante se os delays de I/O fossem eliminados das constraints.